

The Vision Benchmark Module

Image classification: schemas, metrics, execution, and the browser adapters

EdgePilot AI - Team 15

Documentation set, 23 August 2026

4 documents in this volume

src/modules/vision-benchmark/ - image classification benchmarks

Work package: vision benchmark.

What this folder is

The second domain module, with the same three layers as the benchmark module and its own copy of the transport helpers - deliberately a copy, so the two modules can change independently.

Its job is narrower and stricter than the text benchmark's: send an image, get back one label from a closed set, and score the result against a manifest whose SHA-256 is recorded so two runs can be compared only when they measured the same thing.

What happened here

Two changes of a completely different size, and the smaller one mattered more.

The one-character fix. `evidence-store.ts` validated filenames against `/^[a-z0-9._-]+\.json$/` - lowercase only - and then built the filename from an ISO timestamp containing an uppercase T and Z. Every live run failed at the write step. The live path had **never worked, once**. One `.toLowerCase()` is the reason this project has any real measurement at all.

The generalisation. The module now accepts arbitrary label sets, so a user can measure a model against their own images. That required widening six fields - TypeScript interfaces and zod schemas together - and building the prompt from the dataset's own labels rather than a fixed seven. All three pre-existing evidence files were re-parsed afterwards to confirm nothing broke.

The warm-up. The first sample was paying the model-load cost. Median latency across the dataset fell from 1322.1 ms to 529.6 ms once a warm-up classification ran first - the same model, the same images.

Files

File	What it is
<code>index.ts</code>	17 lines. The module's public surface

Subfolders

Layer	What is in it
<code>core/</code>	Types, schemas, metrics, normalisation, the workload prompt
<code>application/</code>	Executor, evaluator, dashboard projection, the provider port
<code>infrastructure/</code>	Server and browser adapters, image processing, manifest loading, evidence writing

Connected folders

- `src/app/vision-benchmark/` - the page.

- `src/app/api/v1/vision-benchmarks/` - the server-side endpoint.
- `src/components/vision/` - the browser path.
- `datasets/vision-benchmark/`, `evidence/vision-benchmark/`, `scripts/vision-benchmark/`, `docs/vision-benchmark/`, `tests/vision-benchmark/`.

Referenced from

These folders point here. Each link below resolves in both directions.

- `datasets/`
- `docs/`
- `src/modules/`

src/modules/vision-benchmark/application/ - running and grading

Work package: vision benchmark.

What this folder is

Four files: the executor that runs a workload sample by sample, the evaluator that grades the result against thresholds, the dashboard projection, and the provider port.

What happened here

executor.ts grew from 159 to 222 lines, all of it for the warm-up classification - and getting that right took three corrections worth recording, because each one is a different class of mistake:

- It **consumed the deterministic test clock**, moving every timing in the suite. Fixed by calling provider.classify() directly rather than the timed classifySample wrapper.
- It **fired on cloud providers**, which meant a billable API call and one extra image sent to a third party on every run. Fixed: local providers only, plus an explicit warmUp?: boolean so a caller can always opt out.
- It **ate the first scripted response** in two existing tests. Those two now opt out with a comment explaining why, and two new tests cover the warm-up properly.

The warm-up's failure is swallowed on purpose: if the throwaway classification errors, the measured run should still proceed.

The executor also threads labels, datasetId and workloadId through, which is what lets a run be about a dataset other than the built-in one.

Files

File	What it is
executor.ts	223 lines. executeVisionBenchmark, the warm-up, the injected clock
evaluator.ts	162 lines. evaluateVisionBenchmark and DEFAULT_VISION_THRESHOLDS - 0.80 accuracy, 0.75 macro-F1, 0.05 maximum invalid-output rate, 0.95 successful-request rate
dashboard.ts	50 lines. toVisionDashboardRow, rankVisionDashboardRows
provider.ts	29 lines. VisionProvider and VisionImageProcessor - the two ports the infrastructure layer implements

Connected folders

- ../core/ - the types, metrics and thresholds.
- ../infrastructure/ - the four provider and two processor implementations of these ports.
- tests/vision-benchmark/ - vision-execution.test.ts.

Referenced from

These folders point here. Each link below resolves in both directions.

- src/modules/vision-benchmark/

src/modules/vision-benchmark/core/ - the vision domain

Work package: vision benchmark.

What this folder is

Five files defining what a vision benchmark **is**: the sample and evidence shapes, the zod schemas that validate them, the metric calculations, the label normaliser, and the prompt.

What happened here

This is where the generalisation to arbitrary datasets was actually done, and where it went wrong first.

`schemas.ts` had `perClass` pinned to `.length(VISION_LABELS.length)` - exactly seven. A dataset with a different class count ran to completion and *then* failed validation, which is the worst possible ordering: all the cost, none of the result. Widening it needed six fields changed across `types.ts` and `schemas.ts` **together**; changing one without the other cost three separate next build round-trips, because Jest type-checks only what it imports.

`workload.ts` grew from 10 lines to 31 for `buildVisionPrompt(labels)`. A deliberate failure run proved it was necessary: with the fixed seven-label prompt and a custom two-class dataset, the result was 0% accuracy and 100% invalid output. Re-run with a prompt built from the dataset's own labels: 100%, passed.

`normalization.ts` is the security-relevant file here. EdgePilot never *acts* on model output, so the threat is not execution - it is output being read as a **verdict**. The defence is exact matching against a closed label set, with anything else recorded as `invalid_output` rather than guessed at.

Files

File	What it is
<code>types.ts</code>	202 lines. <code>VisionBenchmarkSample</code> , <code>VisionBenchmarkEvidence</code> , <code>ClassMetrics</code> , <code>VisionDashboardRow</code> , <code>VISION_LABELS</code> , <code>VISION_DATASET_ID</code> , <code>VISION_WORKLOAD_ID</code>
<code>schemas.ts</code>	278 lines. The zod counterparts, including the manifest and evidence schemas
<code>metrics.ts</code>	171 lines. Exact-match accuracy, macro precision/recall/F1, invalid-output rate, successful-request rate, median and p95 latency, throughput, and the per-class matrix
<code>normalization.ts</code>	60 lines. <code>normalizeVisionLabel</code> - exact matching against the closed set
<code>workload.ts</code>	32 lines. <code>buildVisionPrompt(labels)</code> , <code>VISION_BENCHMARK_PROMPT</code> , and the prompt and workload version constants that make two runs comparable

Connected folders

- `../application/` - the executor and evaluator.
- `../infrastructure/` - what produces the evidence these schemas validate.
- `src/components/vision/` - the upload that calls `buildVisionPrompt` with a user's own labels.
- `evidence/evaluation/` - the injection cases drive `normalizeVisionLabel` directly.

Referenced from

These folders point here. Each link below resolves in both directions.

- `scripts/evaluation/`
- `src/modules/vision-benchmark/`

src/modules/vision-benchmark/infrastructure/ - adapters

Work package: vision benchmark.

What this folder is

Nine files. Two providers and one image processor for the server; one provider and one image processor for the **browser**; plus manifest loading, evidence writing, transport and the run service.

What happened here

The bug that mattered most in the whole project is in this folder. `evidence-store.ts` validated filenames against a lowercase-only pattern and then built them from an ISO timestamp containing T and Z. Nine lines differ; one of them is a `.toLowerCase()`; and before it, no live run had ever managed to write its evidence.

The two browser adapters are new, and they exist for a reason worth stating plainly. `SharpVisionImageProcessor` reads files from disk with a Node library, and `OllamaVisionProvider` builds its base64 with `Buffer`, which does not exist in a browser. For a user's own photographs both are the wrong machine twice over: the images would have to travel to the server, and the model measured would be the server's rather than the user's. So the browser pair prepares images in the tab and talks straight to the user's own Ollama on localhost.

Two details in them are load-bearing:

- **The version string is deliberately different.** `browser-canvas-png-v1`, not `sharp-png-v1`. Canvas and sharp do not resample or encode PNG identically, so runs prepared by each must never be treated as comparable by accident.
- **Determinism is preserved.** `temperature: 0` and `seed: 42`, identical to the server adapter.

Both browser files also hit the same TypeScript trap twice: with `target: es5` and `no downlevelIteration`, spreading a typed array is a hard compile error. Both now use `String.fromCharCode.apply(null, Array.from(...))`.

Files

File	What it is
<code>evidence-store.ts</code>	106 lines. <code>FileVisionEvidenceStore</code> - writes the artefacts. The filename fix lives here
<code>browser-image-processor.ts</code>	181 lines. Canvas resize to 512, <code>crypto.subtle</code> SHA-256, JPEG APP1 EXIF detection. <code>preprocessingVersion = 'browser-canvas-png-v1'</code>
<code>browser-ollama-provider.ts</code>	171 lines. <code>providerName = 'ollama-browser'</code> , posts to the user's own <code>/api/chat</code> from the tab
<code>image-processor.ts</code>	174 lines. <code>SharpVisionImageProcessor</code> - the server-side equivalent
<code>ollama-provider.ts</code>	139 lines. The server-side local vision adapter
<code>gemini-provider.ts</code>	174 lines. The cloud vision adapter
<code>manifest-loader.ts</code>	50 lines. Loads and validates <code>datasets/vision-benchmark/manifest.json</code> , including its SHA-256
<code>run-service.ts</code>	80 lines. <code>runVisionBenchmarkRequest</code> - what the API route calls

`http.ts`

38 lines. The injected `fetch` and `clock`. A deliberate copy of the benchmark module's, not a shared import

Connected folders

- `../application/` - the ports these implement.
- `src/components/vision/` - the upload panel that mounts the two browser adapters.
- `evidence/vision-benchmark/` - what `evidence-store.ts` writes, and why one file there is explicitly un-ignored.
- `datasets/vision-benchmark/` - what `manifest-loader.ts` reads.
- `src/app/api/v1/vision-benchmarks/` - the route that calls `run-service.ts`, and why it sets runtime explicitly.
- `tests/vision-benchmark/` - `vision-infrastructure.test.ts`, `vision-providers.test.ts`.

Referenced from

These folders point here. Each link below resolves in both directions.

- `src/modules/vision-benchmark/`
- `src/modules/vision-benchmark/core/`